

Taylor & Francis
Taylor & Francis Group

Structure and Infrastructure Engineering
Maintenance, Management, Life-Cycle Design and Performance

ISSN: 1573-2479 (Print) 1744-8980 (Online) Journal homepage: www.tandfonline.com/journals/nsie20

A large language model-empowered agent for reliable and robust structural analysis

Jiachen Liu , Ziheng Geng , Ran Cao , Lu Cheng , Paolo Bocchini & Minghui Cheng

To cite this article: Jiachen Liu , Ziheng Geng , Ran Cao , Lu Cheng , Paolo Bocchini & Minghui Cheng (19 Feb 2026): A large language model-empowered agent for reliable and robust structural analysis, Structure and Infrastructure Engineering, DOI: 10.1080/15732479.2026.2630123

To link to this article: https://doi.org/10.1080/15732479.2026.2630123

Published online: 19 Feb 2026.

Submit your article to this journal

Article views: 127

View related articles

View Crossmark data

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=nsie20

STRUCTURE AND INFRASTRUCTURE ENGINEERING
https://doi.org/10.1080/15732479.2026.2630123

Taylor & Francis
Taylor & Francis Group

Check for updates

A large language model-empowered agent for reliable and robust structural analysis

Jiachen Liu[a*], Ziheng Geng[b*], Ran Cao[c], Lu Cheng[d], Paolo Bocchini[e] and Minghui Cheng[b,f]

[a]Department of Electrical and Computer Engineering, University of Miami, Coral Gables, FL, USA; [b]Department of Civil and Architectural Engineering, University of Miami, Coral Gables, FL, USA; [c]College of Civil Engineering, Hunan University, Changsha, China; [d]Department of Computer Science, University of Illinois Chicago, Chicago, IL, USA; [e]Center for Catastrophe Modeling and Resilience, Lehigh University, Bethlehem, PA, USA; [f]School of Architecture, University of Miami, Coral Gables, FL, USA

ABSTRACT

Large language models (LLMs) have exhibited remarkable capabilities across diverse open-domain tasks, yet their application in specialised domains such as civil engineering remains largely unexplored. This paper starts bridging this gap by evaluating and enhancing the reliability and robustness of LLMs in structural analysis of beams. Reliability is assessed through the accuracy of correct outputs under repetitive runs of the same analysis problem, whereas robustness is evaluated based on the performance across varying load and boundary conditions. A benchmark dataset, comprising eight beam structural analysis problems, is created to test the Llama-3.3 70B Instruct model. Results show that, despite an apparent qualitative understanding of structural mechanics, the LLM lacks the quantitative reliability and robustness required for engineering applications. To address these limitations, a shift is proposed that reframes the structural analysis as a code generation task. Accordingly, an LLM-empowered agent is developed that (a) integrates chain-of-thought and few-shot prompting to generate accurate OpenSeesPy code, and (b) automatically executes the code to produce structural analysis results. Experimental results demonstrate that the agent achieves accuracy exceeding 99.0% on the benchmark dataset, exhibiting reliable and robust performance across diverse conditions. Ablation studies highlight the complete example and function usage examples as the primary contributors to the agent's enhanced performance.

ARTICLE HISTORY
Received 28 June 2025
Revised 2 January 2026
Accepted 3 February 2026

KEYWORDS
Automated structural analysis; chain-of-thought; few-shot learning; large language models; LLM agents; reliability; robustness

1. Introduction

Large language models (LLMs) have recently emerged as transformative tools in artificial intelligence, revolutionising how machines comprehend, process, and generate human language. State-of-the-art models, including closed-source model GPT-4o (OpenAI, 2024) and their open-source counterparts like Llama 3.3 (Grattafiori et al., 2024) and DeepSeek-R1 (Guo et al., 2025), have demonstrated exceptional capabilities across a wide range of open-domain tasks, including language translation (Lu et al., 2024; Zhu et al., 2023), text summarisation (Yang et al., 2023; Zhang et al., 2024a), and mathematical reasoning (Ahn et al., 2024; Imani et al., 2023). These advancements have spurred growing interest in specialised, domain-specific applications of LLMs, yielding early promising use cases across fields such as medicine (Arora & Arora, 2023; Thirunavukarasu et al., 2023), law (Graham et al., 2025; Kim et al., 2024), and finance (Lopez-Lira & Tang, 2023; Zhang et al., 2023). These advances have been achieved not by training models from scratch, but by developing effective adaptation techniques that repurpose LLMs for domain-specific tasks (Shen et al., 2024; Wei et al., 2024). These techniques can be categorised into parameter-free methods and parameter-update methods. Parameter-free methods utilise prompt engineering strategies, such as chain-of-thought prompting (Wei et al., 2022), few-shot prompting (Ji et al., 2025; Song et al., 2023), and prompt chaining (Wu et al., 2022). These strategies allow LLMs to learn task formats from expert workflows or limited examples without retraining. Retrieval-augmented generation (RAG) further enhances these methods by incorporating external knowledge bases during inference (Gao et al., 2023). In contrast, parameter-update methods involve modifying partial or full model parameters, including techniques such as full fine-tuning (Lv et al., 2023) and parameter-efficient approaches like low-rank adaptation (LoRA) (Hu et al., 2022). Collectively, these approaches offer great promise in adapting LLMs to domain-specific challenges.

In civil engineering, researchers have also begun to explore the potential of LLMs for a range of specialised applications. One prominent direction involves natural language processing for engineering documentation. For instance, Zhong and Goodfellow (2024) established a construction management corpus and fine-tuned LLMs to support automated compliance checking and asset condition prediction. Similarly, Joffe et al. (2025) developed a retrieval-augmented LLM-based chatbot to answer user

CONTACT Minghui Cheng minghui.cheng@miami.edu Department of Civil and Architectural Engineering, University of Miami, Coral Gables, FL 33146, USA.
*Equal contribution.

© 2026 Informa UK Limited, trading as Taylor & Francis Group

queries regarding building codes and standards. Another line of work leverages LLMs to automate code generation for scientific computing applications. For example, Kim et al. (2025) assessed the capability of ChatGPT to generate finite element code for simulating hydro-mechanically coupled processes. Pandey et al. (2025) developed an LLM agent capable of automatically generating and executing code for computational fluid dynamics (CFD) simulations. Beyond textual and coding tasks, LLMs have also been applied to visual data such as synthesising geological cross-sections from sparse site investigation data (Li & Shi, 2025; Qian & Shi, 2025) and monitoring construction equipment from video frames (Jeoung et al., 2025). Despite these advancements, hallucination remains a critical challenge in LLMs (Huang et al., 2025; Tonmoy et al., 2024). This issue is particularly important in structural engineering, where the consequences for errors can be substantially larger than in other fields. Consequently, a fundamental yet unanswered question arises: how reliable and robust do LLMs perform in structural analysis?

This study aims to take the first step to answer this question by evaluating the performance of a state-of-the-art LLM on simple structural analysis problems that require rigorous understanding of structural mechanics and precise numerical computation. To ensure open-source availability, computational feasibility, and strong instruction-following capabilities, a lightweight LLM, i.e. Llama-3.3 70B Instruct model is selected. A benchmark dataset comprising eight representative beam analysis problems is constructed to assess the model's performance. Results indicate that while Llama-3.3 exhibits a qualitative understanding of structural mechanics, it lacks the reliability and robustness required for engineering applications. To address these limitations, a change of approach is proposed to reframe structural analysis from open-ended text generation tasks to structured code generation tasks. Accordingly, an LLM-empowered agent is developed to integrate chain-of-thought and few-shot prompting techniques to guide LLMs in generating accurate OpenSeesPy (Zhu et al., 2018) code. The generated code is then automatically executed by the agent to provide structural analysis results. Experimental results show that the proposed agent significantly improves the reliability and robustness of LLMs in structural analysis, offering a promising pathway for integrating LLMs into structural engineering practice. The originality of this work lies in the systematic evaluation of the reliability (output consistency across repetitive runs) and robustness (performance across varying input conditions) of LLMs in structural engineering tasks. The proposed LLM-empowered agent is, to the best of our knowledge, the first solution that satisfies the precision requirements of practical engineering applications while supporting local deployment on a standard workstation.

2. Benchmark

2.1. Dataset overview

A benchmark dataset is constructed to evaluate the performance of LLMs on beam structural analysis problems, as illustrated in Figure 1. Beam structural analysis constitutes a fundamental component of structural engineering as it involves foundational concepts such as support conditions, load distribution, and static equilibrium. These concepts form the basis for understanding how external loads are transferred through structural members and how internal forces, including axial forces, shear forces, and bending moments, evolve along the beam. Proficiency in beam structural analysis is essential for civil engineers as it underpins the design and evaluation of more complex structural systems. Herein, as a first step in evaluating the performance of LLMs in structural engineering, the proposed benchmark focuses specifically on the static equilibrium analysis of beams. The primary objective of the benchmark is to solve the reaction forces at the supports of statically determinate beams, including their magnitudes and directions.

To comprehensively assess the performance of LLMs in structural analysis, the benchmark comprises eight representative beam problems incorporating diverse load and boundary conditions. Specifically, two canonical beam types are included: simply supported beams and overhang beams, each with a span of 10 metres. In the case of overhang beams, the roller support is positioned at the mid-span. For each beam type, four distinct load conditions are evaluated: (I) a 10 kN point load, (II) a 10 kN/m uniformly distributed load over a 1-meter segment, (III) a combination of a 10 kN point load and a full-span 10 kN/m uniformly distributed load, and (IV) a combination of a 10 kN point load and a 1-meter 10 kN/m uniformly distributed load. These loads are applied at varying locations along the beam span to reflect a range of realistic design scenarios.

2.2. Workflows to assess reliability and robustness

The assessment of LLM performance in structural analysis focuses on two key dimensions: reliability and robustness, both of which are essential to ensuring that LLMs can serve as trustworthy and confidently deployable tools in engineering practice. Specifically, reliability refers to the accuracy of correct outputs across multiple independent runs of the same analysis problem. This property is particularly important in structural engineering, where even minor inconsistencies can undermine safety and lead to severe consequences. To assess reliability, a standardised workflow is developed, as shown in Figure 2. For each beam structural analysis problem, a textual prompt is provided to specify the beam geometry, support conditions, load conditions, and target outputs. Recognising the sensitivity of LLM performance to prompt formulation, a series of prompt variants are designed and tested in a preliminary step. These variants explore different levels of descriptive detail and structural formatting. The prompt presented here represents the most effective formulation identified through iterative experiments, consistently yielding the highest reliability across various problems. Then, this human-labelled prompt is input into the LLM for 500 independent runs. The reliability of the model is quantified through the proportion of correct responses across these repeated runs, as follows:

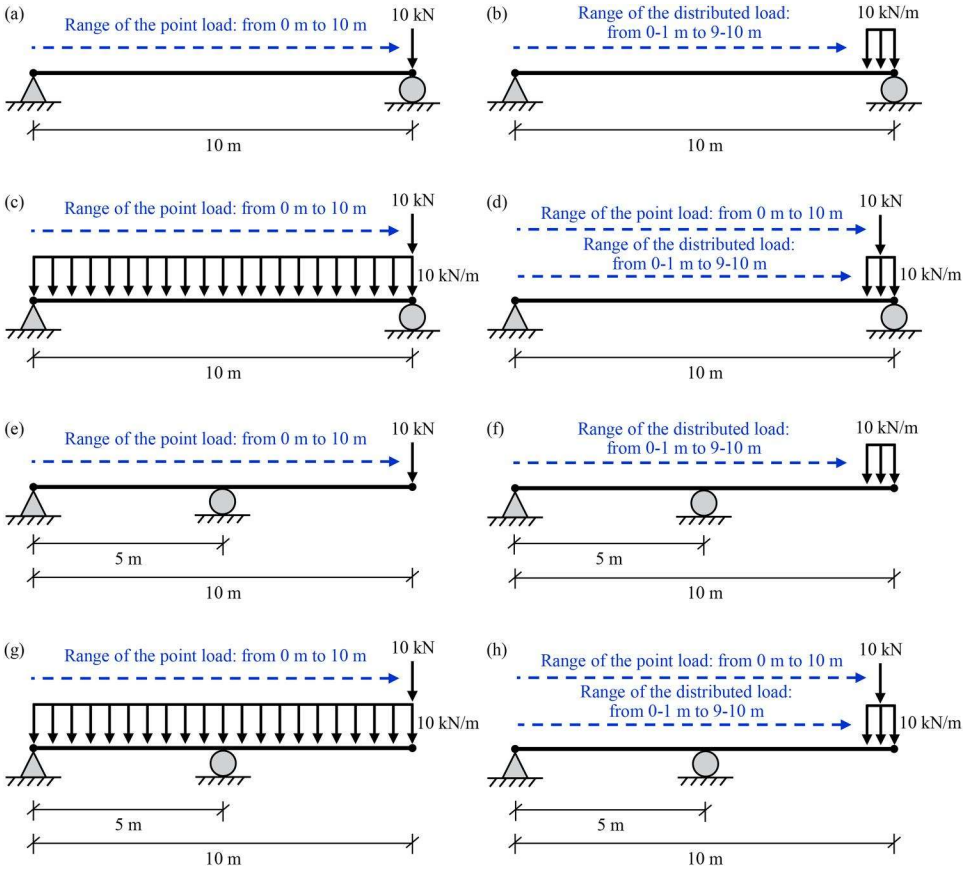


Figure 1. A benchmark dataset comprising eight representative beam structural analysis problems.

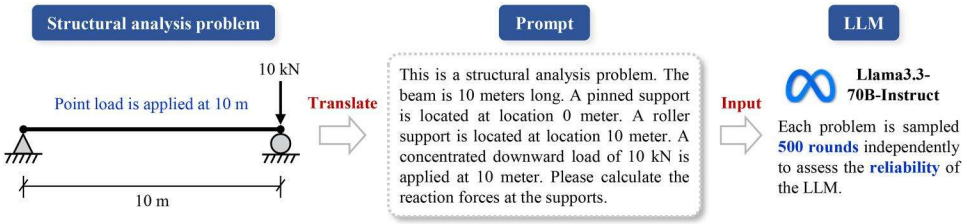


Figure 2. Workflow to assess the reliability of LLMs for structural analysis problems.

$$\text{Reliability} = \frac{N_{\text{correct}}}{N_{\text{total}}} \tag{1}$$

where N_{correct} represents the number of correct outputs and N_{total} denotes the total number of independent runs, i.e. 500 runs.

Robustness is defined as the model's ability to maintain its performance under variations in input parameters (Yu et al., 2025). Herein, the robustness is evaluated with respect to reliability and the variations refer to the changes in load and support conditions. This attribute is vital for ensuring the model's adaptability to a wide range of real-world scenarios. To assess robustness, each structural analysis problem in the benchmark dataset (Figure 1) is evaluated under a series of varied load conditions, wherein the applied load is incrementally moved from the right end to the left end of the beam in 1-meter intervals, as demonstrated in Figure 3. Each variation is studied independently, and for each variation, a corresponding textual prompt is provided and submitted to the LLM for 500 independent runs. To ensure a bounded and interpretable robustness metric, a normalised robustness score is defined with reference to the inverse transformation of the coefficient of variation (CV) (Hoes et al., 2009), as follows:

$$\text{Robustness} = (1 + \text{CV})^{-1} = \left(1 + \frac{\sigma_R}{\overline{R}}\right)^{-1} \tag{2}$$

where $\overline{R}$ and σ_R denote the mean and standard deviation of the reliability under varying load conditions, respectively. This formulation ensures that the robustness value lies within the interval $(0, 1]$, where higher values indicate stronger robustness.

2.3. Performance of Llama-3.3 70B Instruct model

Following the evaluation workflow described above, the reliability and robustness of the Llama-3.3 70B Instruct model

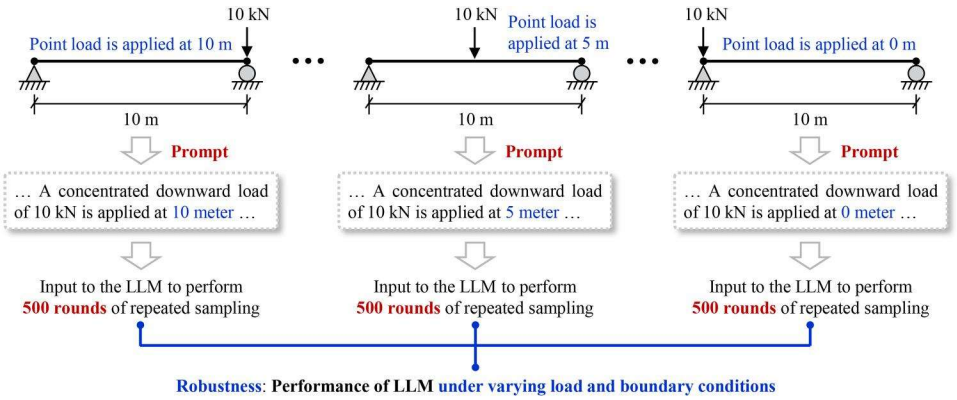


Figure 3. Workflow to assess the robustness of LLMs for structural analysis problems.

are assessed using the benchmark dataset. The results are presented in Figure 4, where each subplot corresponds to a specific combination of load and boundary conditions. In each case, the load is applied at different locations along the beam span, producing a reliability curve. Each point on the curve represents the reliability of the model at a given load location, computed from 500 independent runs. Specifically, several key patterns are observed in the results of simply supported beams: (I) The LLM demonstrates strong performance under single-load scenarios, achieving average reliability values of 87.1% for point loads and 93.6% for localised distributed loads, as shown in Figure 4(a,b). (II) The performance of LLM degrades notably as the load conditions become increasingly complex. When a point load is combined with a full-length distributed load or a localised distributed load, the average reliability drops to 75.8% and 73.1%, respectively, as depicted in Figure 4(c,d). (III) The LLM exhibits high reliability when loads are applied within the span of the beam. However, its robustness deteriorates significantly when the loads are positioned at the supports, as evidenced by the sharp reliability dips at the beam ends across Figure 4(a–d).

In contrast to the simply supported beams, the overhang beam exhibits a distinctly different reliability pattern. As illustrated in Figure 4(e–h), the reliability of the LLM declines dramatically when the load is applied outside the region bounded by the supports, i.e. in the cantilevered region, compared to when the load is applied within the supported span. Particularly, for the case involving a combination of point loads and full-length distributed loads, the value of reliability even falls below 10.0%, as shown in Figure 4(g). Additionally, two patterns observed in simply supported beams are also evident in the overhang beams. (I) The average reliability of the LLM decreases with increasing load complexity. This can be demonstrated by comparing the performance between the single-load scenarios in Figure 4(e,f) and the combined load cases in Figure 4(g,h). (II) A reduction in reliability is also observed when the load is applied at support locations, as shown by the lower reliability values at the leftmost and mid-span positions across Figure 4(e–h).

To qualitatively assess the LLM's understanding of the fundamental principles underlying structural analysis tasks, three representative incorrect responses generated by the Llama-3.3 70B Instruct model are presented in Figure 5. In example 1, the LLM correctly identified static equilibrium as the basic principle and formulated the vertical force equilibrium equation. However, it failed to incorporate the moment equilibrium equation and instead relied on an improper assumption that the load was equally shared by the pinned and roller supports. This assumption neglects the fact that the load was applied directly at the roller support, leading to a fundamental error in reasoning and a wrong numerical solution. In example 2, the LLM correctly applied both the vertical force equilibrium and the moment equilibrium equations. Nevertheless, a computational mistake occurred during the algebraic manipulation of equations, resulting in an incorrect numerical answer despite an otherwise sound approach. In example 3, the LLM successfully formulated and solved both the force and moment equilibrium equations, obtaining the correct magnitudes of the support reactions. However, it incorrectly reported the direction of the reaction force as downward instead of upward, indicating a conceptual misunderstanding in the interpretation of mechanical equilibrium. Collectively, these examples indicate that while the LLM demonstrates a qualitative understanding of structural mechanics, it lacks high reliability and robustness required for practical engineering applications.

3. A large language model-empowered agent for structural analysis

3.1. Reframe the task: from text generation to code generation

To improve the reliability and robustness of LLM in structural analysis problems, a change of approach is proposed that reframes the task from text generation to code generation, as shown in Figure 6. This shift is driven by two primary motivations. First, LLM predictions inherently rely on next-token prediction. For text generation tasks, it typically involves high degrees of freedom and uncertainty due to the open-ended nature of natural language. Specifically, natural language is context-dependent and often lacks strict rules, resulting in a vast latent space where a wide range of outputs can be syntactically valid but semantically vague or incorrect. In contrast, code

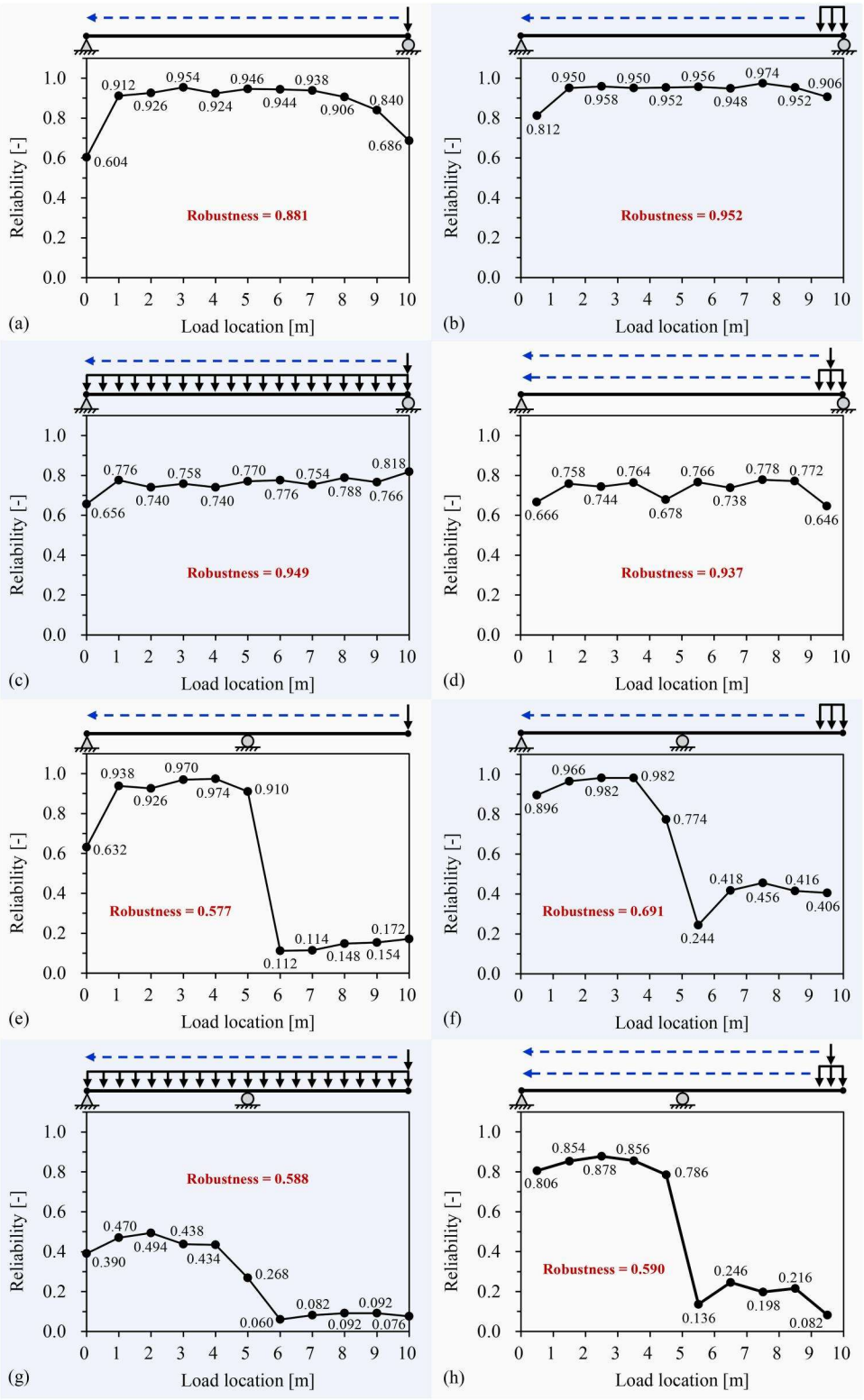


Figure 4. Reliability and robustness of Llama-3.3 70B instruct model on the benchmark dataset.

generation operates within a framework defined by rigorous syntactic and semantic rules, which significantly constrains the output space and reduces predictive uncertainty. This structured nature enables LLMs to generate output that conforms to the conventions of programming languages, thereby enhancing consistency and reliability. Existing studies have demonstrated that LLMs tend to exhibit higher accuracy in code generation tasks compared to text generation tasks due to the reduced ambiguity (Li et al., 2023; Roziere et al., 2023). Therefore, reframing structural analysis as a code generation task allows LLMs to capitalise on their strengths in modelling structured sequences, thereby producing more reliable and robust outputs for engineering problem-solving.

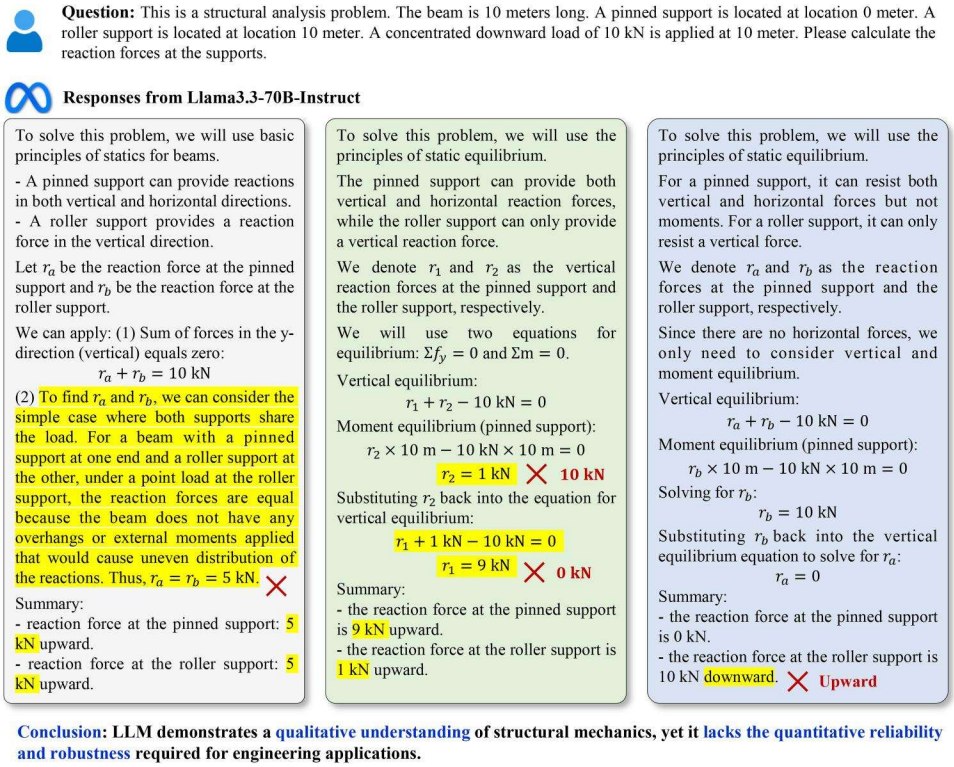


Figure 5. Illustrative examples of incorrect solutions produced by the Llama-3.3 70B Instruct model in structural analysis tasks.

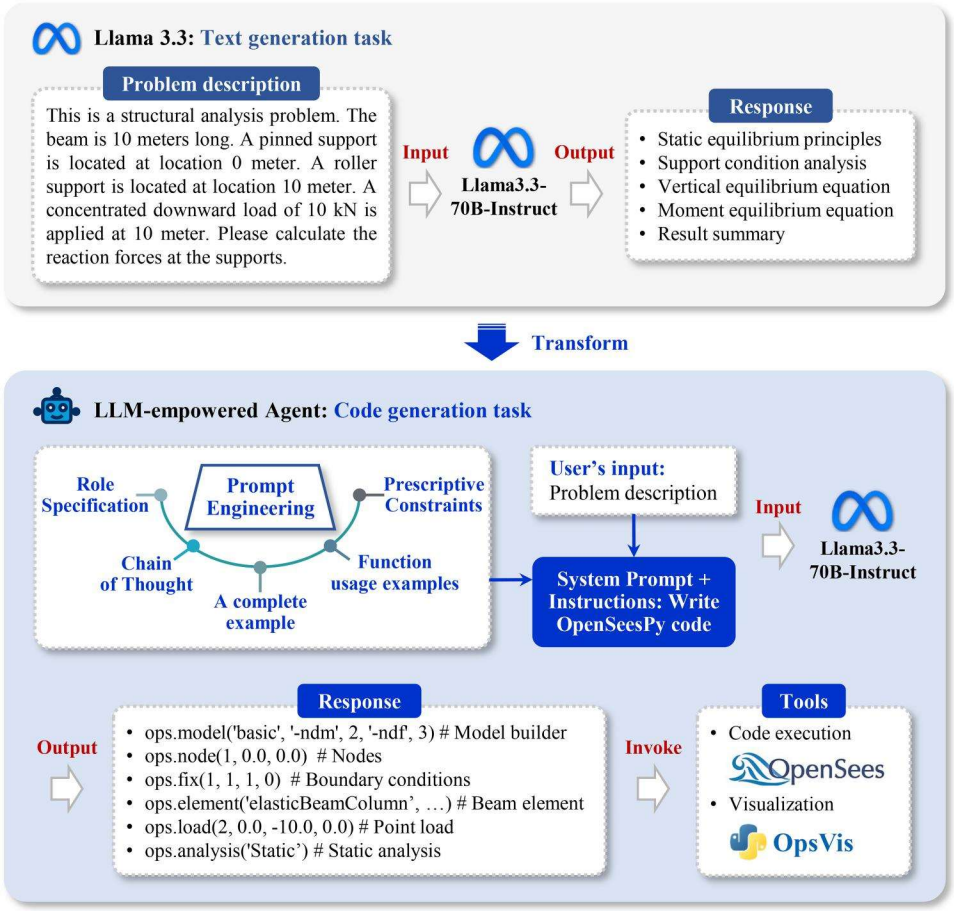


Figure 6. A shift from text generation to code generation for structural analysis tasks using an LLM-empowered agent.

Second, code generation offers a more scalable approach for adapting LLMs to domain-specific tasks (Zhang et al., 2024b). Rather than requiring LLMs to internalise specialised domain knowledge and perform complex symbolic computations, which may lead to hallucinated or incorrect output due to their probabilistic nature, it is more effective to integrate LLMs with established computational tools. This strategy aligns with current practices in civil engineering, where professionals routinely rely on dedicated software platforms to address real-world problems. By positioning the LLM as a language-to-action bridge, the proposed approach synergistically leverages the superior capability of LLMs in code generation and the reliability and robustness of domain-specific computational software. Such integration is expected to reduce the cognitive burden on users and promote the automation of task analysis and execution. Additionally, the modularity and reusability of codes facilitate a scalable workflow, enabling LLMs to break down complex, high-level problems into smaller, programmable subtasks. This capacity can enhance the adaptability and practical utility of LLMs in practical engineering applications.

3.2. Agent architecture

To enable the automated generation and execution of structural analysis codes, an LLM-empowered agent is developed, specifically designed to integrate with the OpenSeesPy platform. OpenseesPy is an open-source Python library that supports finite element modelling for a variety of structural types, including beams, trusses, and frames (Zhu et al., 2018). For visualisation, OpenSeesPy is integrated with the OpsVis package (Kokot, 2024), which provides graphical representations of structural behaviour, thereby enhancing result interpretability and user interaction through intuitive visual feedback.

The architecture of the agent incorporates a dedicated prompt engineering layer designed to enhance the capabilities of LLMs in generating accurate and domain-specific OpenSeesPy code, as illustrated in Figure 6. This layer comprises five components. The predefined prompts are integrated with the user-provided input, typically a natural language problem description, to construct the system prompt. The system prompt is then input into the LLM, along with explicit instructions to generate OpenSeesPy code corresponding to the described structural analysis task. Upon generating the code, the agent proceeds to invoke OpenSeesPy for numerical simulation and OpsVis for result visualisation. This modular workflow allows each component to operate in a well-defined and interpretable pipeline. By leveraging the strengths of both LLMs in code generation and domain-specific computational solvers in structural analysis, the agent provides a reliable and interpretable framework for automating structural analysis.

3.3. Prompt design

To improve the accuracy, reliability, and domain alignment of LLM outputs in beam analysis problems, a structured prompt template is proposed, as shown in Figure 7. This template consists of five key components: role specification, chain of thought, a complete example, function usage examples, and prescriptive constraints. The overarching design philosophy leverages two complementary prompting strategies: chain of thought reasoning and few-shot learning. The former encourages the LLM to reason through intermediate steps, decomposing complex structural problems into manageable sub-tasks. The latter provides concrete examples, ranging from complete code generation sequences to modular snippets for key functions, to ground LLM's responses in domain-relevant logic and syntax. This dual mechanism mitigates semantic drift, enhances output consistency, and reduces the risk of hallucination, thereby facilitating the transformation of the LLM from a general-purpose language model into a task-specialised assistant for structural analysis.

Each component in the prompt template serves as a distinct role in guiding the generative behaviour of the LLM. Specifically, the role specification component assigns a clear identity to the LLM, instructing it to act as a structural

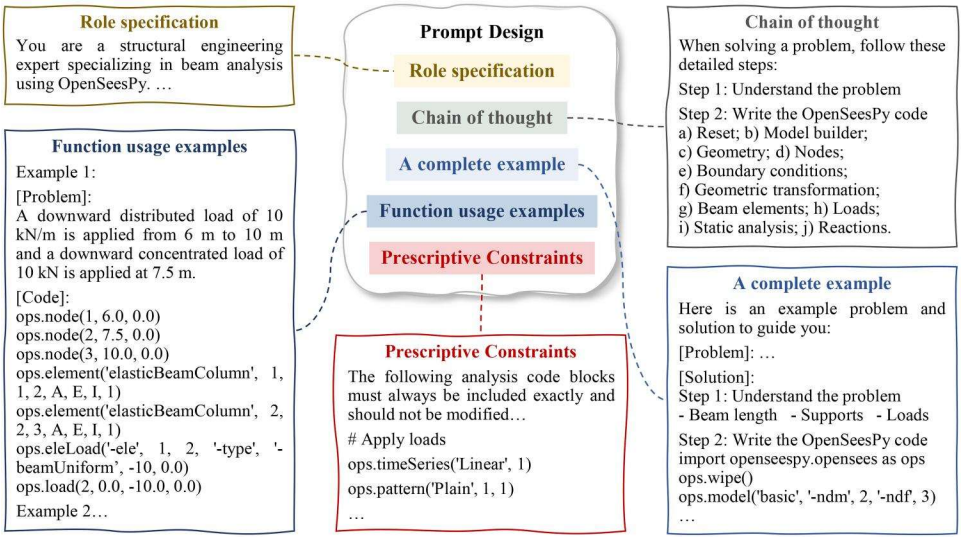


Figure 7. A prompt template for reliable and robust structural analysis.

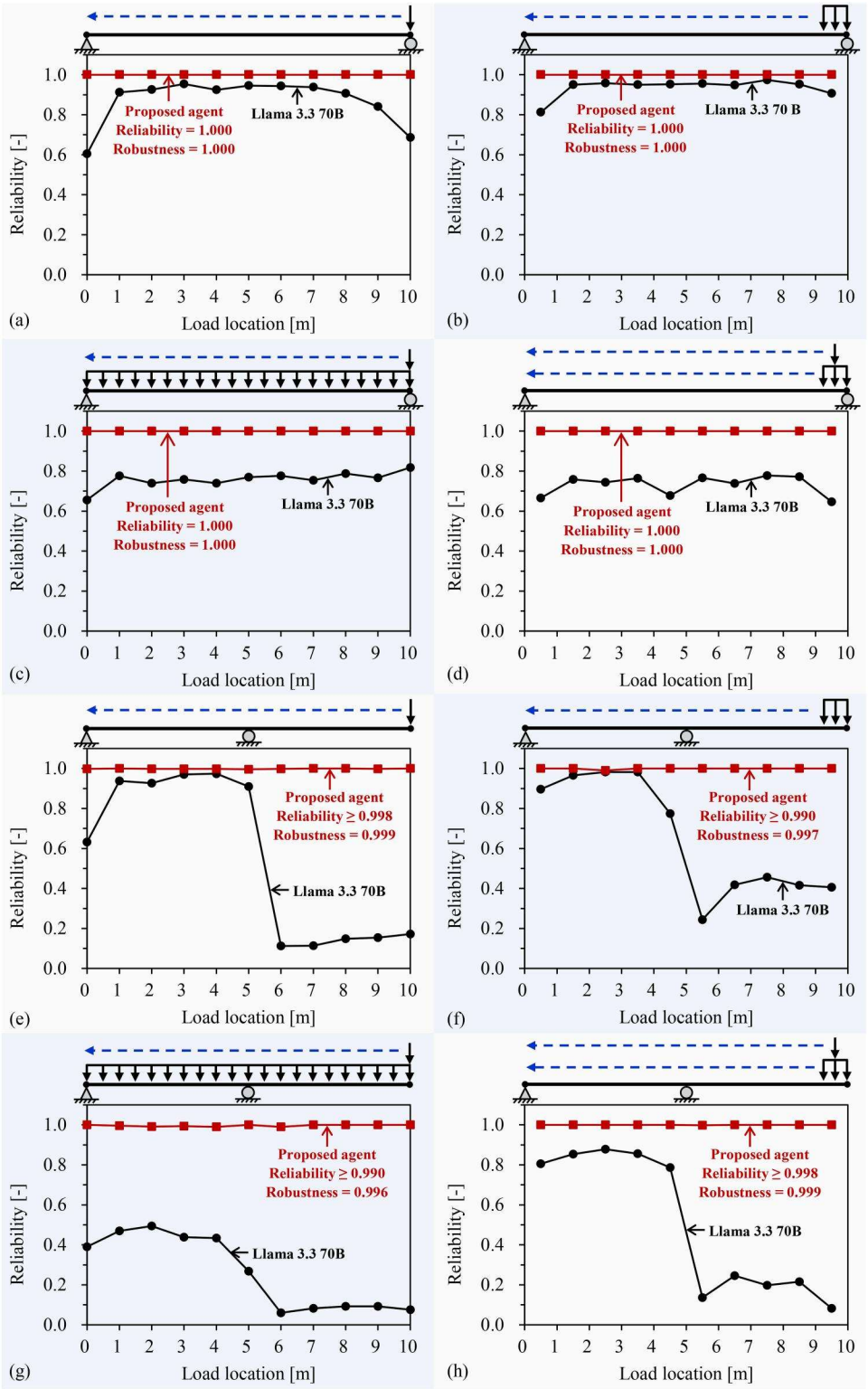


Figure 8. Reliability and robustness of the proposed LLM-empowered agent on the benchmark dataset.

engineering expert with specialised knowledge in beam analysis using OpenSeesPy. This ensures that the model activates relevant domain knowledge and maintains a consistent tone and context throughout the response. The chain of thought is divided into two stages. First, the LLM is prompted to interpret the problem by identifying key aspects including beam geometry, boundary conditions, loading conditions, and analysis objectives. Second, the LLM is prompted to outline the logical sequence for code construction, emulating the reasoning process of a human engineer. Following the reasoning step, an example is introduced to provide a full instance of code generation for a simple beam problem. Note that this example demonstrates the complete workflow, including chain of thought reasoning and executable code writing. It serves as a reference from which the LLM can learn the logic and structure for similar tasks.

To further enhance task-specific performance, a set of function usage examples are incorporated to address common weaknesses in LLM-generated outputs. These challenges primarily involve the definition of nodes, elements, and load cases, particularly under complex loading conditions such as concentrated and distributed loads applied concurrently. The function usage examples begin with instructional tutorials on critical OpenSeesPy functions, providing detailed explanations of input parameters related to beam elements, boundary conditions, and load applications. This is followed by three applied examples that demonstrate how to construct models in which a concentrated load lies within the span of a distributed load. These examples help the LLM learn to correctly identify nodes, segment beam elements, and apply complex loading configurations in a robust and generalisable manner. Finally, prescriptive constraints are incorporated to regulate code formatting, where mandatory code blocks are specified to prevent unauthorised modifications. These constraints enhance code compatibility with downstream analysis tools and minimise the risk of execution errors.

4. Results and discussion

4.1. Performance of the proposed LLM-empowered agent

During the performance assessment experiments, the Llama-3.3 70B Instruct model was accessed via the Lambda API. Across the benchmark dataset, the average inference time per query ranges from 19.6 to 25.5 s. With an API price of $0.12 per million input tokens and $0.30 per million output tokens, the average cost per problem is approximately $0.0008. The responses are labelled as correct if the calculated reaction forces match the reference solutions in both direction and magnitude. A ± 1% numerical tolerance is applied to consider acceptable deviations in predicted values.

The performance of the proposed LLM-empowered agent is assessed using the benchmark dataset in terms of reliability and robustness. As illustrated in Figure 8, the proposed agent consistently achieves reliability over 0.990 and robustness over 0.996 across a range of load and boundary conditions, significantly outperforming the baseline Llama-3.3 70B Instruct model. Particularly, the agent exhibits substantial improvements in reliability under conditions where the baseline model's performance degrades, such as under point loads or localised distributed loads applied at support locations or in cantilevered regions. These improvements yield flat reliability curves, demonstrating the strong robustness of the proposed agent in structural analysis tasks. The enhanced performance can be attributed to two key strategies. First, by reframing structural analysis as a code-generation task, the problem is transformed into a structured and standardised workflow. This mitigates ambiguity or inaccuracy in natural language reasoning, thereby improving the robustness of the agent across varied conditions. Second, the incorporation of dedicated prompt templates provides reasoning guidance and concrete examples, which enhances the consistency and reliability of the agent's outputs over repetitive runs. Together, these strategies enable a reliable and robust LLM-empowered agent for automated structural analysis. Representative execution examples of the proposed agent are provided in Figure 9.

Additionally, a detailed analysis of the agent-generated incorrect cases is conducted, identifying two primary error types. The first type involves the misapplication of boundary conditions in cantilever beam problems. For these cases, the correct configuration involves a pinned support at the left end and a roller support at the mid-span, leaving the right end as a free cantilever. However, the agent occasionally hallucinates an additional roller support at this free end. This is a typical form of LLM hallucination, where the model defaults to more common structural configurations learned during its training, rather than strictly adhering to the prescribed conditions in the problem statement. The second error type involves the inaccurate application of loads, particularly under combined load conditions. For example, when tasked with applying a distributed load from 1 to 2 metres and a point load at 1.5 metres, the agent extends the range of the distributed load to 0–2 metres. Although function usage examples have been demonstrated in the prompt template, the agent occasionally struggles to precisely parse the combined load conditions, leading to intermittent inaccuracies in load application.

4.2. Generalisation performance on extended statically determinate beams

Beyond the benchmark dataset, the proposed LLM-empowered agent is further tested on a series of more complicated beam analysis problems to assess its generalisation capabilities and robustness under varied structural conditions. As illustrated in Figure 10, these test cases introduce variations in load locations, load directions, support types, and support locations, all of which differ from the conditions presented in the benchmark dataset. Specifically, three representative problems are designed. (a) An overhang beam with a pinned support at 0 metres and a roller support at 5 metres. The beam is subjected to two upward loads: a point load at 9 metres and a uniformly distributed load spanning from 7.5 to 9.5 metres. (b) An overhang beam with a pinned support at 0 metres and a roller support at 7 metres. The beam is subjected to two downward loads: a point load at 8.5 metres and a uniformly distributed load ranging from 8.2 to 9.7 metres. (c) A cantilever beam with a fixed support at 0 metres. The beam is subjected to two downward loads: a point load at 9 metres and a uniformly distributed load spanning from 2.5 to 7.5 metres.

For each test case, a problem description in natural language is provided to the agent as input. In problems (a) and (b), the agent is tasked with solving the reaction forces at the pinned and roller supports, whereas in problem (c), the agent is instructed to determine both the reaction forces and bending moment at the fixed support. To assess the reliability of the agent under these generalised conditions, each problem is executed independently over 500 runs. The

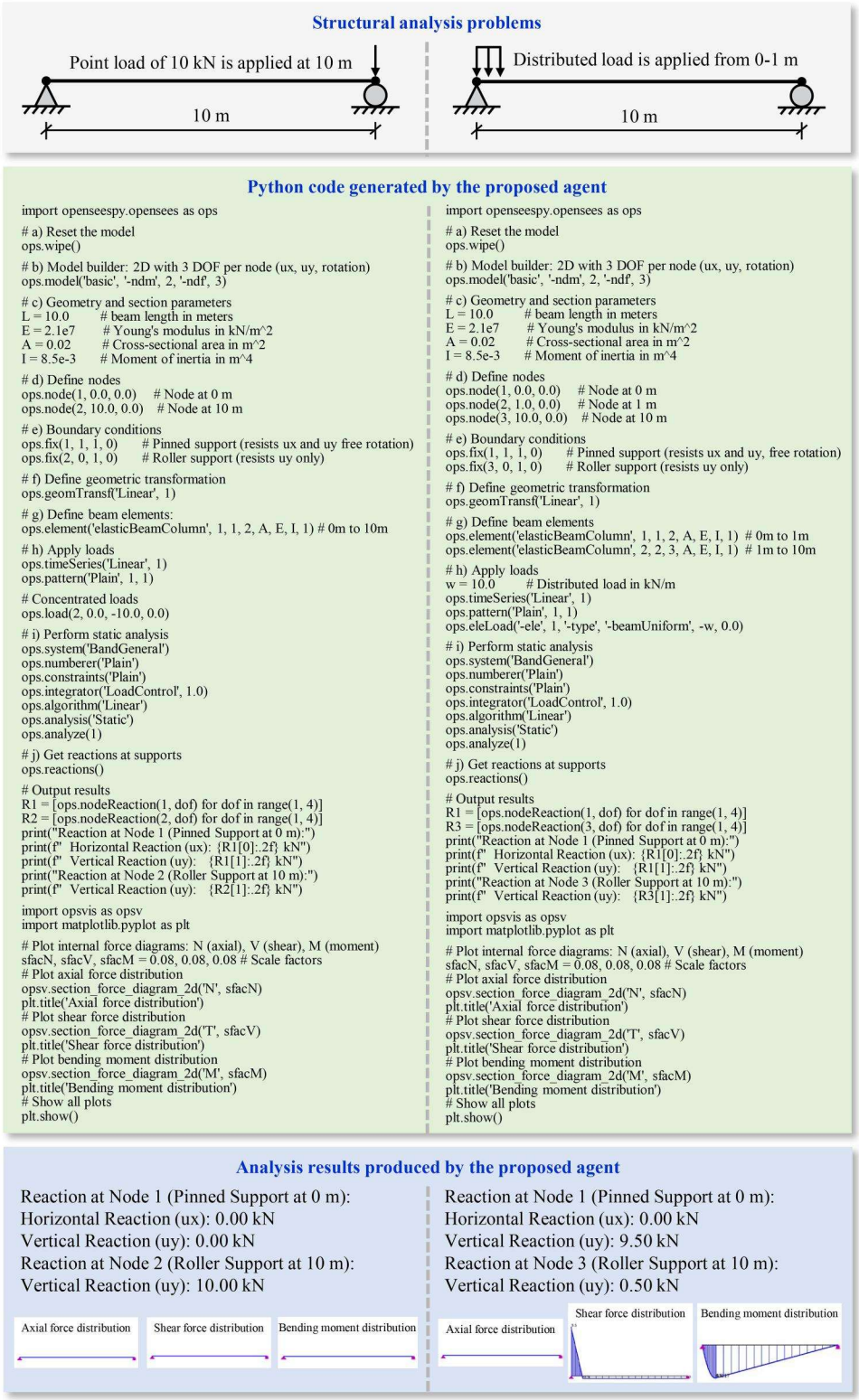
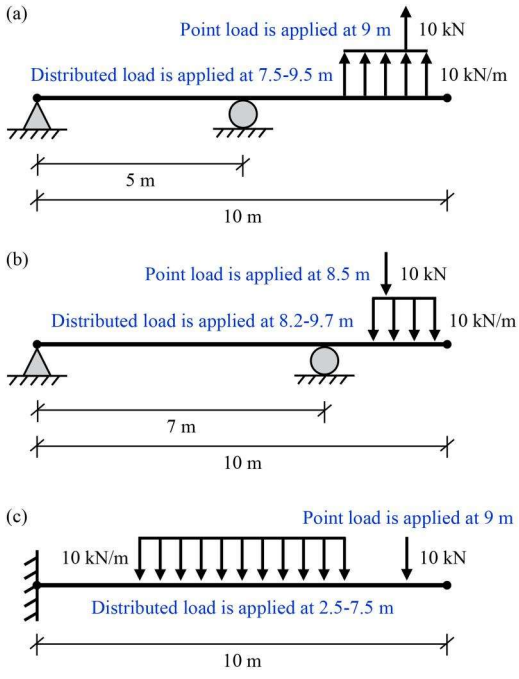


Figure 9. Execution examples of the proposed LLM-empowered agent.

results indicate that the proposed agent maintains strong reliability across three extended scenarios, achieving reliability of 100%, 99.8%, and 99.6% for problems (a), (b), and (c), respectively. These findings highlight the agent's capability to generalise to novel beam analysis scenarios while maintaining reliable and robust performance. Such generalisation ability is essential for real-world applications in structural engineering, where each structure may present distinct loading and boundary constraints.

OpenSeesPy codes produced by the LLM-empowered agent are automatically executed to compute the reaction forces and bending moments. The resulting structural

responses are subsequently visualised using the OpsVis package, as shown in Figure 11. These visualisations include the finite element models along with the corresponding axial force, shear force, and bending moment diagrams for each case. The constructed finite element models exhibit strong alignment with domain-specific physical expectations, accurately reflecting the intended node configurations, loading conditions, and support placements. This consistency confirms that the proposed agent generates syntactically correct and valid codes. In summary, the visualisation module in the agent can significantly enhance the practical utility of the proposed agent by providing engineers with graphical representations of structural behaviour, facilitating model validation, error checking, and decision-making in real-world applications. Moreover, the graphical output goes in the direction of providing an explanation for the results, and caters to the desire of explainability in the use of LLMs.



Metric	Task (a)	Task (b)	Task (c)
Reliability	1.000	0.998	0.996

Figure 10. Generalisation performance of the proposed agent on extended beam analysis tasks.

4.3. Extension to statically indeterminate structures

To demonstrate the broader applicability of the proposed agent, the evaluation is extended to statically indeterminate structures, including beams and portal frames. These problems are representative of practical engineering scenarios where hand calculations become time-consuming, labour-intensive, and error-prone. Automating these tasks highlights the advantages of the proposed agent in improving modelling efficiency and reducing manual effort in real-world engineering workflows.

For the statically indeterminate beams, the proposed agent is evaluated on five representative problems, as illustrated in Figure 12. These cases encompass a range of structural configurations, from single-span propped cantilevers to three-span and five-span continuous beams. The boundary conditions involve various combinations of fixed, pinned, and roller supports, while the loading patterns include uniformly distributed loads, concentrated loads, and their combinations. Despite the increased structural complexity, the agent achieves error-free performance with a reliability of 1.000 across all five cases. These results validate the agent's ability to accurately interpret natural language descriptions and apply consistent modelling syntax to define nodes, elements, loads, and boundary conditions. By integrating the reasoning capabilities of LLMs with these domain-specific tools, the agent effectively generalises from statically determinate to statically indeterminate beams.

Additionally, two single-bay, single-story portal frames are developed to further assess the agent's capability in analysing statically indeterminate structures, as illustrated in Figure 13. The detailed configurations are as follows. Frame (a) consists of a 6-meter by 6-meter geometry with pinned supports at the base and is subjected to a 20 kN rightward point load at the top-left corner. Frame (b) spans 8 metres in width and 6 metres in height, with fixed support at the base. It is subjected to a combined loading condition comprising a 20 kN rightward point load at the top-left corner and a uniformly distributed downward load of 10 kN/m along the top beam. The agent demonstrates strong

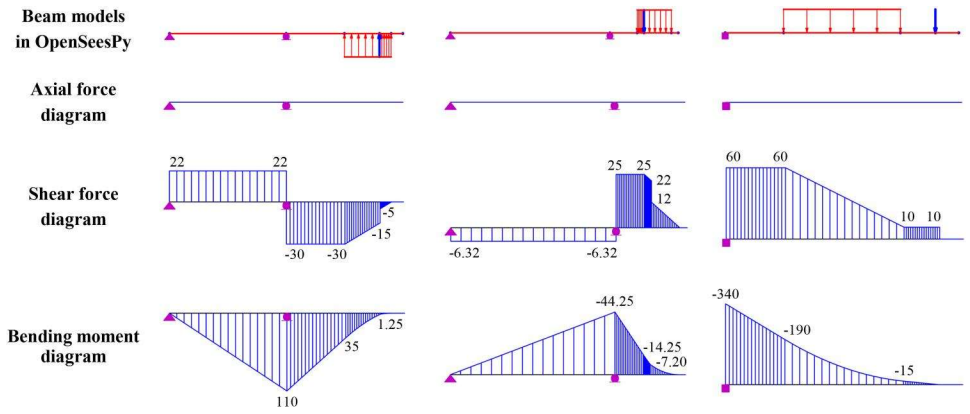
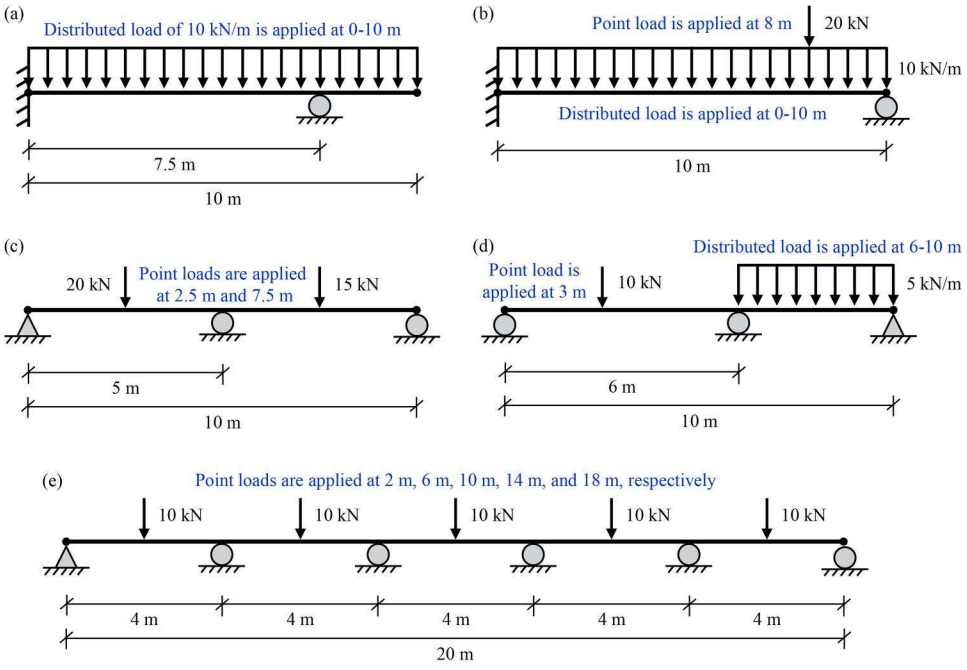


Figure 11. Visualisation of the proposed agent for extended beam analysis problems.



Metric	Task (a)	Task (b)	Task (c)	Task (d)	Task (e)
Reliability	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**

Figure 12. Performance of the proposed agent on representative statically indeterminate beams.

reliability in performing structural analysis for both cases, achieving scores of 1.000 and 0.986, respectively. The visualisation of the analysis results is shown in Figure 14, including both internal force distributions and deformation profiles. These results indicate that the agent can effectively apply the specialised syntax for structural analysis, thereby exhibiting robust generalisation beyond simple beams.

4.4. Ablation experiments on prompt components

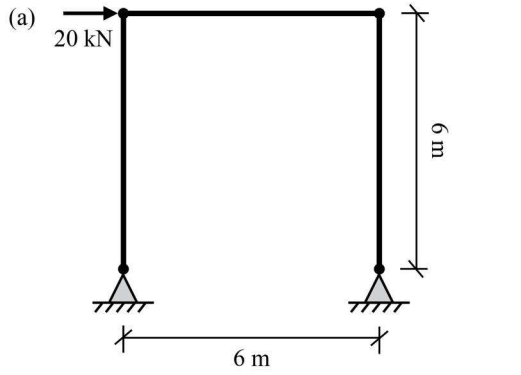
To systematically assess the contribution of individual components in the structured prompt template, a series of ablation experiments are conducted using the extended beam analysis tasks in subsection 4.2. In each experiment, one component is removed from the prompt template to isolate its effect, resulting in four prompt configurations: without chain of thought reasoning, without the complete example, without function usage examples, and without prescriptive constraints. Each configuration is applied to all three extended tasks and the reliability of the agent is assessed over 500 independent runs.

The results of these ablation experiments are presented in Table 1. It is shown that the complete example emerges as the most influential component. Its removal causes the reliability of the agent to dramatically drop to 0% across all three extended tasks. This behaviour is attributed to LLM's inability to infer the full modelling workflow, resulting in missing or disordered code segments and subsequent execution errors. Function usage examples also play a vital role in maintaining the agent's performance. When these examples are excluded, the reliability of the agent declines significantly to 2.6%, 29.6%, and 62.3% for the three tasks, respectively. A detailed inspection of the outputs reveals that most errors stem from incorrect node placement and improper application of loads. These findings highlight that the function usage examples improve the LLM's ability to learn domain-specific modelling strategies under complex load conditions, thereby enhancing the agent's generalisation capability.

Additionally, the exclusion of the chain of thought or prescriptive constraints leads to more moderate performance degradation, as illustrated in Table 1. Specifically, without the chain of thought component, the agent still achieves high reliability of 100% and 99.8% in the first two overhang beam tasks. However, its performance decreases to 78.4% in the cantilever beam case. Given that the cantilever case is not included in the prompt template, this observation suggests that chain of thought reasoning helps activate latent domain knowledge within the LLM, thus improving the generalisation and robustness of the agent. Similarly, removing prescriptive constraints leads to reliability of 93.6%, 97.0%, and 55.6% across the three tasks. Errors in this condition are primarily due to the omissions of essential code blocks, emphasising the role of hard-coded constraints in enforcing structural consistency and minimising execution failure. Collectively, the above results underscore the critical contribution of all prompt components to the reliable and robust performance of the proposed agent in structural analysis applications.

5. Practical deployment and implications

The proposed agent can be integrated into existing engineering workflow with minimum setup. Its deployment involves two main components: (a) installation of structural analysis packages, including OpenSeesPy and OpsVis, and (b) deployment of the Llama-3.3 70B Instruct model, which is a publicly available LLM that supports both local and cloud-based access. For local deployment, the model can be hosted on a workstation with at least 40 GB VRAM. For cloud-based deployment, the model can be accessed via collaborative platforms such as Lambda. Once deployed, users can utilise the prompt template as the system prompt to reproduce the results presented in this study. By describing a structural problem in natural language, the agent automatically generates executable OpenSeesPy code for structural analysis.



Metric	Task (a)	Task (b)
Reliability	1.000	0.986

Figure 13. Performance of the proposed agent on single-bay, single-story portal frames.

The proposed agent can provide tangible benefits across multiple aspects of structural engineering workflows. First, the agent can automatically generate and execute code for structural analysis of beams and simple frames. The automated workflow eliminates the need for manual coding or repetitive software interactions, significantly improving efficiency and reducing error. For example, one of the authors conducted a manual coding experiment on problem (e) in Figure 12, which required 10 min and 43 s. In contrast, the agent generates the corresponding code in just 55 s, demonstrating an improvement in speed by 10 times. Second, the agent enables direct translation from natural language descriptions into executable code, enhancing input flexibility and accessibility. This capability lowers the entry barrier for students and non-specialists who may lack expertise with specialised software. The resulting graphical output, such as deformed shapes and internal force diagrams, provide intuitive visualisations of structural behaviour, supporting educational and training applications. Third, the methodology validated on beams provides a scalable foundation for future extension to complex structural systems, such as multi-story frames and irregular geometries. In such cases, the modelling complexity grows substantially due to

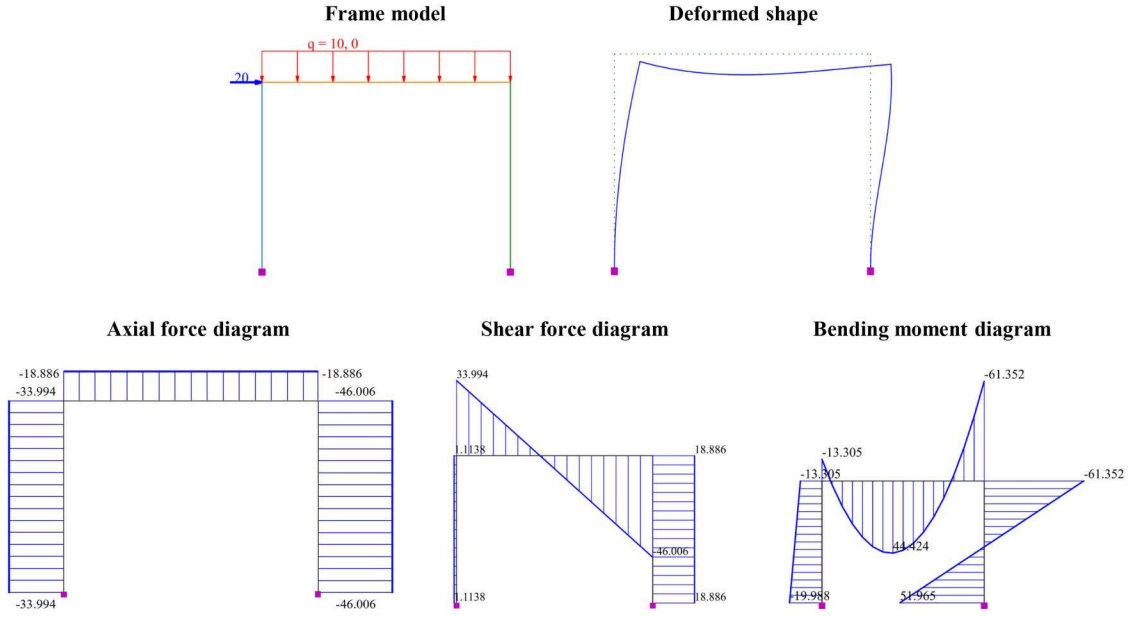


Figure 14. Visualisation of the proposed agent for structural analysis of single-bay, single-story portal frames.

Table 1. Ablation experiments on prompt components.

Prompt configuration	Reliability: Task (a)[a]	Reliability: Task (b)[a]	Reliability: Task (c)[a]	Robustness
Proposed prompt template	1.000	0.998	0.996	0.998
w/o chain of thought	1.000	0.998	0.784	0.882
w/o complete example	0.000	0.000	0.000	–
w/o function usage examples	0.026	0.296	0.623	0.513
w/o constraints	0.936	0.970	0.556	0.781

[a]Tasks (a), (b), and (c) correspond to extended beam analysis problems shown in Figure 10.

the need to ensure element connectivity and spatial consistency.

6. Conclusions

This paper represents a first step towards assessing and enhancing the reliability and robustness of large language models (LLMs) in structural engineering, with a focus on the static equilibrium analysis of beams. The assessment results reveal that while the Llama-3.3 70B Instruct model demonstrates a qualitative understanding of structural mechanics, it lacks the quantitative reliability and robustness required for practical engineering applications. To address these limitations, the structural analysis task is shifted from text generation to code generation. Accordingly, an LLM-empowered agent is developed to automatically generate and execute OpenSeesPy and OpsVis code based on natural language input. It shows that the proposed agent achieves strong reliability exceeding 99.0% and robustness exceeding 99.6% across all benchmark problems, significantly outperforming the baseline model.

Beyond these findings, several forward-looking insights emerge from the results:

- The ablation experiments reveal the central roles of the complete example and function usage examples in enhancing the agent's performance. These examples enable the LLM to interpret semantic descriptions, learn logical reasoning, and internalise coding syntax from representative cases. This observation underscores the great potential of generalising the proposed approach to other tasks. By constructing a comprehensive portfolio of task-specific examples, the capabilities of LLMs can be extended to tackle analogous tasks that share similar logical and code patterns. Such a scalable framework provides a promising pathway towards reliable and robust solutions for a wide range of engineering applications.
- The agent's ability to produce visual output not only enhances the interpretability of LLMs but also opens new avenues for interactive human–LLM collaboration. Specifically, the visual output can serve as immediate feedback for analysts, allowing engineers to interactively construct complex structural configurations through a series of incremental modifications, starting from simple base cases. This progressive workflow leverages the domain-specific knowledge of human experts to identify and correct potential errors in LLM-generated codes, thereby improving the usability, adaptability, and practical integration of LLM-based tools in engineering applications.
- While this study focuses on structured natural language input, the agent occasionally hallucinates additional loads or boundary conditions that are not specified in the problem description. This limitation highlights the potential of employing vision-language models (VLMs), which can jointly process textual descriptions and graphical representations of structures. Additionally, as structural configurations become increasingly complex, such as 2D frames or 3D assemblies, textual descriptions may be difficult to construct and prone to ambiguity. Therefore, leveraging VLMs to interpret both visual and textual inputs may improve model understanding of structural configurations, representing a promising direction for future research and practical deployment.
- Despite the reliable and robust performance, the proposed agent is limited to analysing beams and simple portal frames. It lacks the spatial reasoning capabilities required to model complex structural systems, such as multi-span, multi-story, or geometrically irregular frames. Future work should focus on enhancing the agent's ability to handle complex geometries and automate code generation for such structures. Additionally, although LLM outputs are inherently stochastic, the agent does not incorporate uncertainty quantification to estimate the confidence of its predictions. Future work should investigate the source of LLM hallucinations and develop methods to quantify predictive uncertainty.
- Although the prompt template has mitigated some instances of hallucination, the agent may still produce incorrect results, particularly when problem descriptions deviate from the expected format. Therefore, users are encouraged to follow the standardised input template provided. In cases of incorrect outputs, human intervention remains essential. Graphical output enables engineers to quickly identify and correct modelling errors. This underscores the collaborative role of LLMs and human expertise in the human-in-the-loop framework.

Author contributions

CRediT: **Jiachen Liu**: Data curation, Investigation, Software, Writing – original draft; **Ziheng Geng**: Data curation, Investigation, Software, Writing – original draft; **Ran Cao**: Methodology, Supervision, Writing – review & editing; **Lu Cheng**: Methodology, Supervision, Writing – review & editing; **Paolo Bocchini**: Methodology, Writing – review & editing; **Minghui Cheng**: Conceptualization, Methodology, Supervision, Writing – review & editing.

Disclosure statement

No potential conflict of interest was reported by the author(s).

ORCID

Minghui Cheng http://orcid.org/0000-0002-8983-5148

Data availability statement

Some or all data, models, or code that support the findings of this study are available from the corresponding author upon reasonable request.

References

Ahn, J., Verma, R., Lou, R., Liu, D., Zhang, R., & Yin, W. (2024). Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*

Arora, A., & Arora, A. (2023). The promise of large language models in health care. *Lancet* , *401*(10377), 641. https://doi.org/10.1016/S0140-6736(23)00216-7

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv Preprint, arXiv:2312.10997 2:1.

Graham, S. G., Soltani, H., & Isiaq, O. (2025). Natural language processing for legal document review: Categorising deontic modalities in contracts. *Artificial Intelligence and Law*, *33*(1), 79–100. https://doi.org/10.1007/s10506-023-09379-2

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. (2025). Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*

Hoes, P., Hensen, J. L., Loomans, M. G., de Vries, B., & Bourgeois, D. (2009). User behavior in whole building simulation. *Energy and Buildings*, *41*(3), 295–302. https://doi.org/10.1016/j.enbuild.2008.09.008

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. (2022). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, *43*(2), 1–55. https://doi.org/10.1145/3703155

Imani, S., Du, L., & Shrivastava, H. (2023). Mathprompter: Mathematical reasoning using large language models. *arXiv preprint arXiv:2303.05398*.

Jeoung, J., Jung, S., & Hong, T. (2025). Zero-shot framework for construction equipment task monitoring. *Computer-Aided Civil and Infrastructure Engineering*, *40*(21), 3218–3238. https://doi.org/10.1111/mice.13506

Ji, B., Duan, X., Zhang, Y., Wu, K., & Zhang, M. (2025). Zero-shot prompting for LLM-based machine translation using in-domain target sentences. *IEEE Transactions on Audio, Speech and Language Processing*, *33*, 251–261. https://doi.org/10.1109/taslp.2024.3519814

Joffe, I., Felobes, G., Elgouhari, Y., Talebi Kalaleh, M., Mei, Q., & Chui, Y. H. (2025). The framework and implementation of using large language models to answer questions about building codes and standards. *Journal of Computing in Civil Engineering*, *39*(4), 05025004. https://doi.org/10.1061/JCCEE5.CPENG-6037

Kim, M.-Y., Rabelo, J., Babiker, H. K. B., Rahman, M. A., & Goebel, R. (2024). Legal information retrieval and entailment using transformer-based approaches. *The Review of Socionetwork Strategies*, *18*(1), 101–121. https://doi.org/10.1007/s12626-023-00153-z

Kim, T., Yun, T. S., & Suh, H. S. (2025). Can ChatGPT implement finite element models for geotechnical engineering applications? *International Journal for Numerical and Analytical Methods in Geomechanics*, *49*(6), 1747–1766. https://doi.org/10.1002/nag.3956

Kokot, S. (2024). *OpsVis documentation*. Retrieved June 7, 2025, from https://sewerynkokot.github.io/opsvis/

Li, H., & Shi, C. (2025). Few-shot learning of geological cross-sections from sparse data using large language model. *Geodata and AI*, *2*, 100010. https://doi.org/10.1016/j.geoai.2025.100010

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. (2023). Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*.

Lopez-Lira, A., & Tang, Y. (2023). Can ChatGPT forecast stock price movements? Return predictability and large language models. SSRN Electronic Journal *arXiv:2304.07619*. https://doi.org/10.2139/ssrn.4412788

Lu, Y., Zhu, W., Li, L., Qiao, Y., & Yuan, F. (2024). LLaMAX: Scaling linguistic horizons of LLM by enhancing translation capabilities beyond 100 languages. *arXiv preprint arXiv:2407.05975*.

Lv, K., Yang, Y., Liu, T., Gao, Q., Guo, Q., & Qiu, X. (2023). Full parameter fine-tuning for large language models with limited resources. *arXiv preprint arXiv:2306.09782*.

OpenAI. (2024). *GPT-4o: OpenAI's multimodal flagship model*. Retrieved May 21, 2025, from https://openai.com/index/hello-gpt-4o/

Pandey, S., Xu, R., Wang, W., & Chu, X. (2025). OpenFOAMGPT: A retrieval-augmented large language model (LLM) agent for OpenFOAM-based computational fluid dynamics. *Physics of Fluids*, *37*(3), 035120. https://doi.org/10.1063/5.0257555

Qian, Z., & Shi, C. (2025). Large language model-empowered paradigm for automated geotechnical site planning and geological characterization. *Automation in Construction*, *173*, 106103. https://doi.org/10.1016/j.autcon.2025.106103

Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Shen, J., Tenenholtz, N., Hall, J. B., Alvarez-Melis, D., & Fusi, N. (2024). Tag-LLM: Repurposing general-purpose LLMs for specialized domains. *arXiv preprint arXiv:2402.05140*.

Song, C. H., Sadler, B. M., Wu, J., Chao, W.-L., Washington, C., & Su, Y. (2023). *LLM-Planner: Few-shot grounded planning for embodied agents with large language models* [Paper presentation]. Proceedings of the 2023 IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 2986–2997). IEEE Computer Society. https://doi.org/10.1109/ICCV51070.2023.00280

Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F., & Ting, D. S. W. (2023). Large language models in medicine. *Nature Medicine*, *29*(8), 1930–1940. https://doi.org/10.1038/s41591-023-02448-8

Tonmoy, S., Zaman, S., Jain, V., Rani, A., Rawte, V., Chadha, A., & Das, A. (2024). A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313 6*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*, 24824–24837.

Wei, J., Zhuo, L., Fu, X., Zeng, X., Wang, L., Zou, Q., & Cao, D. (2024). Drugrealign: A multisource prompt framework for drug repurposing based on large language models. *BMC Biology*, *22*(1), 226. https://doi.org/10.1186/s12915-024-02028-3

Wu, T., Terry, M., & Cai, C. J. (2022). AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (pp. 1–22). Association for Computing Machinery.

Yang, X., Li, Y., Zhang, X., Chen, H., & Cheng, W. (2023). Exploring the limits of ChatGPT for query or aspect-based text summarization. *arXiv preprint arXiv:2302.08081*.

Yu, T., Jing, Y., Zhang, X., Jiang, W., Wu, W., Wang, Y., Hu, W., Du, B., & Tao, D. (2025). Benchmarking reasoning robustness in large language models. *arXiv preprint arXiv:2503.04550*.

Zhang, B., Yang, H., Zhou, T., Ali Babar, M., & Liu, X.-Y. (2023). *Enhancing financial sentiment analysis via retrieval augmented large language models* [Paper presentation]. Proceedings of the Fourth ACM International Conference on AI in Finance (pp. 349–356). Association for Computing Machinery. https://doi.org/10.1145/3604237.3626866

Zhang, T., Ladhak, F., Durmus, E., Liang, P., McKeown, K., & Hashimoto, T. B. (2024a). Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics*, *12*, 39–57. https://doi.org/10.1162/tacl_a_00632

Zhang, W., Li, Y., Dong, Z., Wu, Y., Zhou, Y., Wang, D., Xing, S., Zhou, C., & Shen, D. (2024b). Renaissance of literate programming in the era of LLMs: Enhancing LLM-based code generation in large-scale projects. *arXiv preprint arXiv:2502.17441*.

Zhong, Y., & Goodfellow, S. D. (2024). Domain-specific language models pre-trained on construction management systems corpora. *Automation in Construction*, *160*, 105316. https://doi.org/10.1016/j.autcon.2024.105316

Zhu, M., McKenna, F., & Scott, M. H. (2018). Openseespy: Python library for the opensees finite element framework. *SoftwareX*, *7*, 6–11. https://doi.org/10.1016/j.softx.2017.10.009

Zhu, W., Liu, H., Dong, Q., Xu, J., Huang, S., Kong, L., Chen, J., & Li, L. (2023). Multilingual machine translation with large language models: Empirical results and analysis. *arXiv preprint arXiv:2304.04675*.